

UNIVERSIDAD TECNOLÓGICA DEL NORTE DE AGUASCALIENTES

Tecnología y valores, educación sin límites

CARRERA Y GRUPO:

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN E INNOVACIÓN DIGITAL

7 A

MATERIA:

FUNDAMENTOS DE INTELIGENCIA ARTIFICIAL

DOCENTE:

GERARDO MARTÍNEZ MARTÍNEZ

ACTIVIDAD:

IMPLEMENTACIÓN DE UN PROGRAMA EN PYTHON, DOCUMENTACIÓN Y EXPOSICIÓN.

ELABORADO POR:

FERNANDA KATZENI BARRALES LÓPEZ, DULCE YOSELYN PÉREZ MARTÍNEZ, DEISY MARLEN LÓPEZ GASPAR, SANTIAGO JASSO ESPINO.

FECHA: 10/11/2026

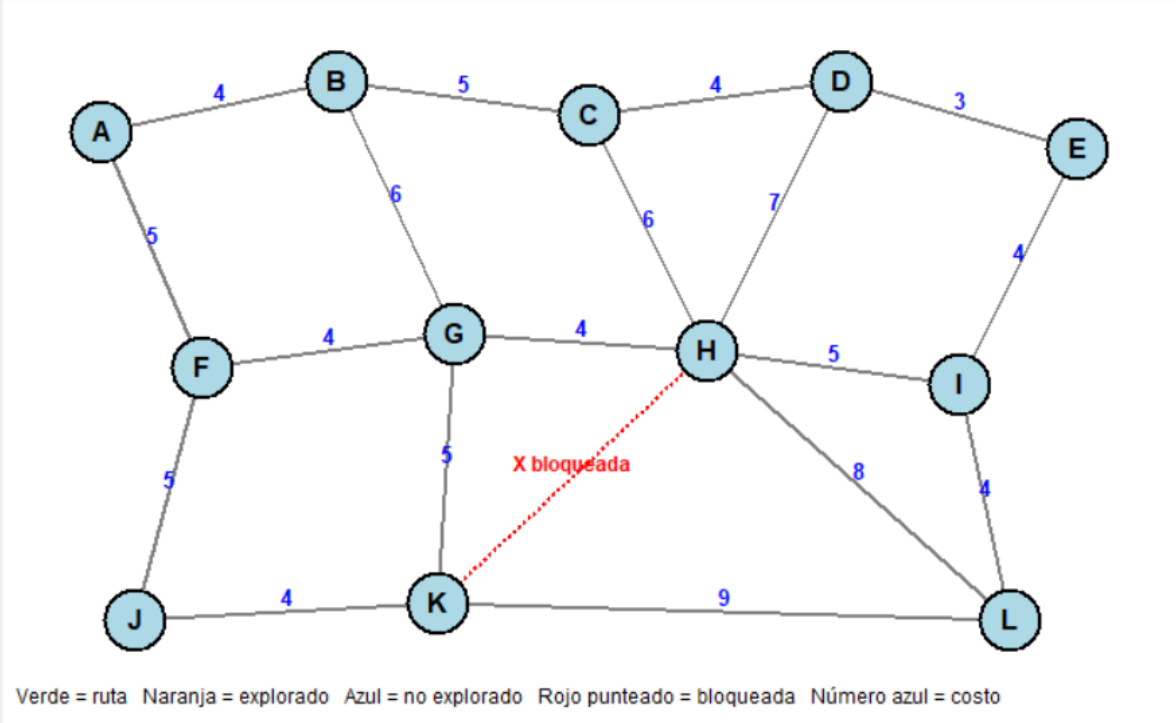
INTRODUCCIÓN

Una empresa de logística necesita un sistema que permita encontrar rutas entre diferentes ubicaciones. Algunas rutas tienen mayor costo que otras y existen caminos bloqueados. Además, la empresa necesita encontrar soluciones eficientes cuando el número de posibilidades es demasiado grande.

Para ello, se desarrolló un prototipo que plasma el recorrido por medio de grafos, el cual se recorre de manera experimental implementando 4 tipos de algoritmos de búsqueda, DFS, BFS, Greedy Best First Search y A*.

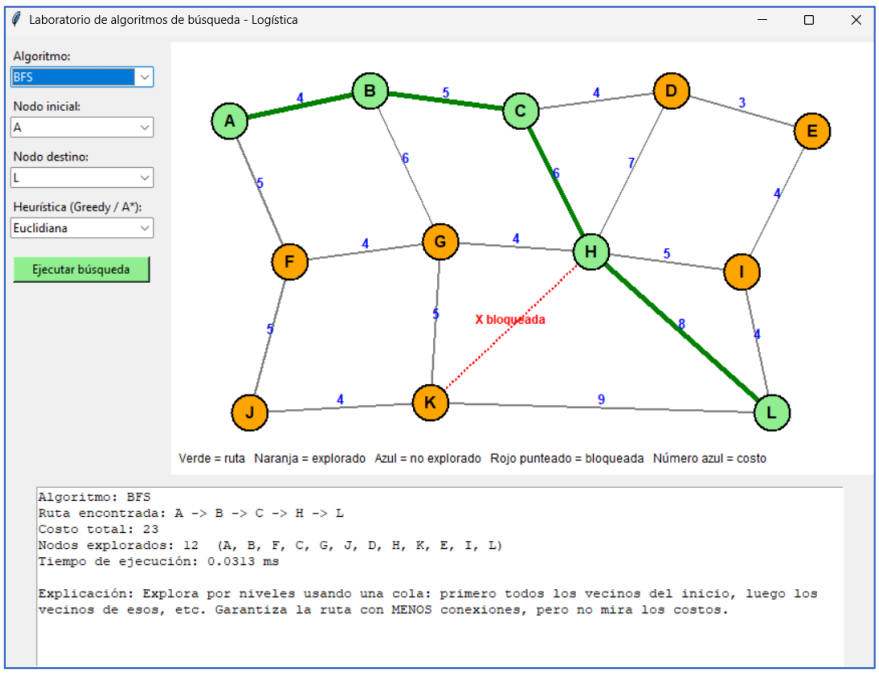
MAPA REPRESENTADO MEDIANTE UN GRAFO

El mapa se representa con un grafo no dirigido: cada ciudad es un nodo y cada carretera es una conexión con un costo. Se cumplen los requisitos: 12 nodos (mínimo 10), 18 conexiones activas (mínimo 16), costos distintos, inicio A, destino L, varias rutas alternativas y una conexión bloqueada (H-K) que el programa no puede usar.

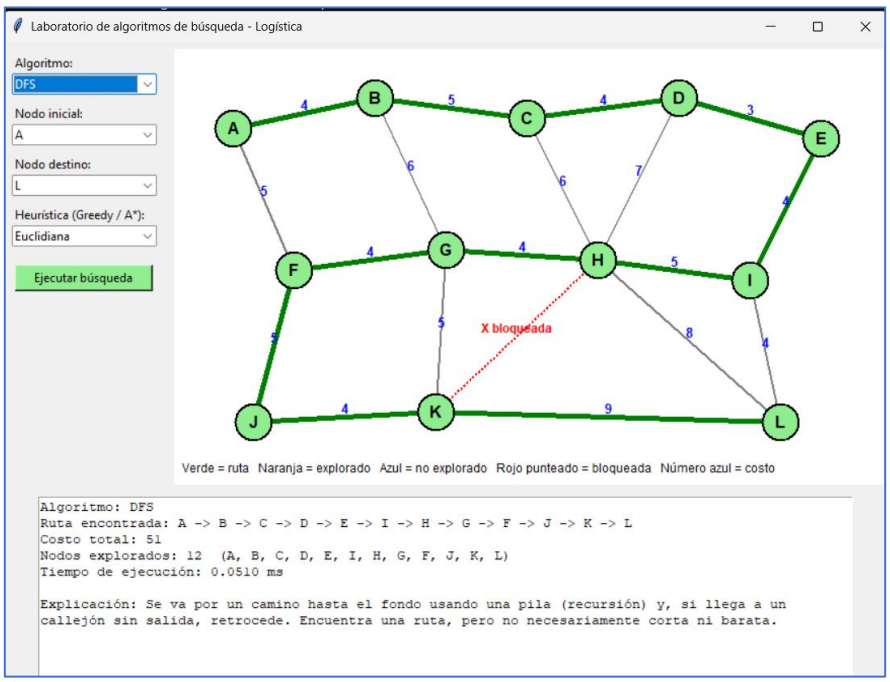


ALGORITMOS IMPLEMENTADOS

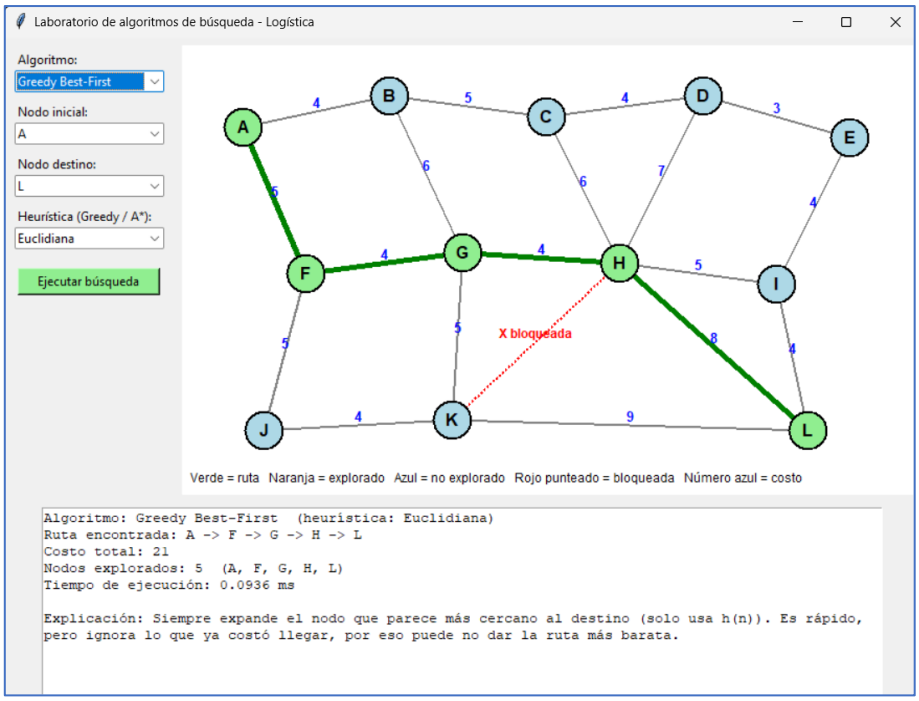
BFS (búsqueda en anchura). Explora por niveles usando una cola: primero todos los vecinos del inicio, luego los vecinos de esos, etc. Garantiza la ruta con MENOS conexiones, pero no mira los costos.



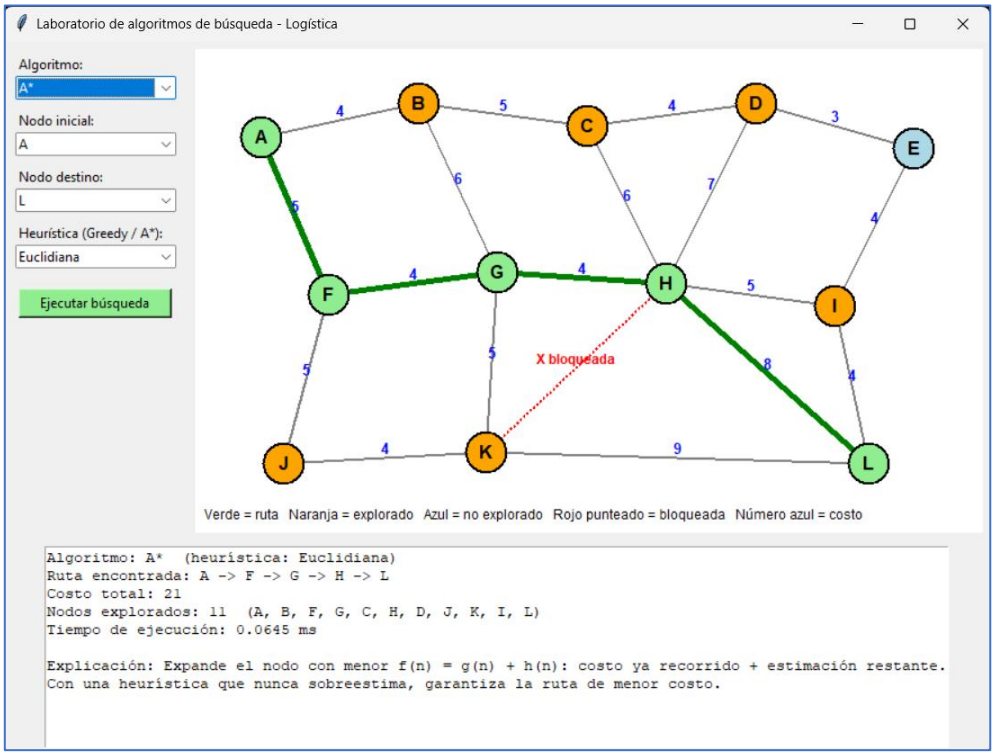
DFS (búsqueda en profundidad). Se va por un camino hasta el fondo usando una pila (recursión) y, si llega a un callejón sin salida, retrocede. Encuentra una ruta, pero no necesariamente corta ni barata.



Greedy Best-First. Siempre expande el nodo que parece más cercano al destino (solo usa h(n)). Es rápido, pero ignora lo que ya costó llegar, por eso puede no dar la ruta más barata.



A*. Expande el nodo con menor f(n) = g(n) + h(n): costo ya recorrido + estimación restante. Con una heurística que nunca sobreestima, garantiza la ruta de menor costo.



En todos los casos, el número de nodos explorados cuenta los nodos que el algoritmo sacó de su estructura (cola, pila o cola de prioridad) y revisó, incluyendo el destino. El tiempo se mide con time.perf_counter().

Análisis y comparación

Algoritmo	Ruta o solución	Costo	Nodos explorados	Tiempo de ejecución (ms)
BFS	A – B – C – H – L	23	12	0.0313
DFS	A – B – C – D – E – F – G – H – I – J – K – L	51	12	0.0510
Greedy Best-First	A – F – G – H – L	21	5	0.0936
A*	A – F – G – H – L	21	11	0.0645

PREGUNTAS

a) ¿Qué algoritmo encontró la ruta de menor costo?

A* y Greedy empataron con costo 21 (ruta A - F - G - H - L), frente a 23 de BFS y 51 de DFS. Sin embargo, solo A* garantiza el menor costo. Greedy lo logró en este caso por suerte: si se busca de A a I, Greedy da costo 20 (A - B - C - H - I) mientras que A* da 18 (A - F - G - H - I).

b) ¿Por qué BFS y DFS pueden encontrar rutas diferentes?

Porque recorren el grafo en distinto orden. BFS usa una cola y avanza por niveles, así que la primera vez que llega al destino lo hace con el menor número de conexiones (4 conexiones, costo 23). DFS usa una pila y se mete lo más profundo posible por el primer vecino; encontró una ruta de 11 conexiones y costo 51, porque tomó el primer camino que llevaba al destino sin comparar alternativas. Además ninguno de los dos mira los costos.

c) ¿Qué sucede cuando se modifica la heurística de Greedy y A*?

Se probaron tres heurísticas (A a L):

Con la heurística cero, Greedy pierde toda su guía y explora casi todo el mapa (ruta de costo 23), mientras que A* se comporta como un algoritmo de costo uniforme: sigue encontrando el costo 21 pero explorando más nodos. Con la heurística inflada (sobreestima la distancia), A* explora menos nodos pero deja de ser óptimo (costo 22 en lugar de 21).

Conclusión: la calidad de la heurística cambia la velocidad y, en A*, solo se garantiza el óptimo si la heurística nunca sobreestima.

d) ¿Qué ventaja ofrece A* frente a Greedy?

A* considera el costo ya recorrido g(n) además de la estimación h(n), por lo que no se deja engañar por un nodo que parece cercano pero al que se llegó por un camino caro. Greedy solo usa h(n): es más rápido (en este mapa exploró 5 nodos contra 11 de A*), pero puede dar rutas más caras, como en el ejemplo A a I. A* garantiza la ruta óptima con una heurística admisible. e) ¿Qué algoritmo elegirían para un sistema real de logística y por qué? A*. En logística importa el costo real (distancia, tiempo, peajes, combustible), y A* encuentra la ruta más barata y es más eficiente que explorar todo el mapa, porque la heurística (por ejemplo, la distancia en línea recta entre ciudades) lo orienta hacia el destino. BFS ignora los costos, DFS puede dar rutas larguísimas (costo 51 aquí) y Greedy no garantiza que la ruta sea buena. Además, A* maneja bien las carreteras bloqueadas, ya que basta con quitar esa conexión del grafo.

CONCLUSIÓN

Cada algoritmo responde a un objetivo distinto: BFS minimiza conexiones, DFS solo busca una ruta, Greedy busca rapidez y A* equilibra costo y estimación para encontrar la ruta más barata. Aprendimos que la heurística es clave y que elegir el algoritmo depende de lo que se quiera optimizar.